# GSoC 2026 Application

## The Mifos Initiative - Mifos X Smart Contract & Loan Agreement Summarization with LLMs

**Applicant:** Wanjing Yang | wanjingyang29@gmail.com **Slack:** Wanjing Yang **Timezone:** PST (UTC-7)
**GitHub:** https://github.com/Niuniu-Li-229/2026-GSoC-Mifos/tree/main/Contract%20Summarization%20with%20LLMs

---

## 1. Project idea:

Mifos X Smart Contract & Loan Agreement Summarization with LLMs (Medium, 175 hours)

---

## 2. Briefly cover what you expect to complete in this project and when, including a detailed description and project plan:

### 2.1 Problem Statement

Loan agreements in microfinance are often written in dense legal language that is inaccessible to low-literacy borrowers — creating real over-indebtedness risk at the point of origination. Key terms like declining-balance interest, total cost of credit, and default consequences are buried in compliance boilerplate rather than communicated clearly.

Mifos X already captures all the structured data needed to explain a loan clearly, but there is no tool that converts it into plain-language client communication automatically. This project fills that gap.

### 2.2 Proposed Solution

A Python-based LLM summarization service embedded into the Mifos X workflow that:

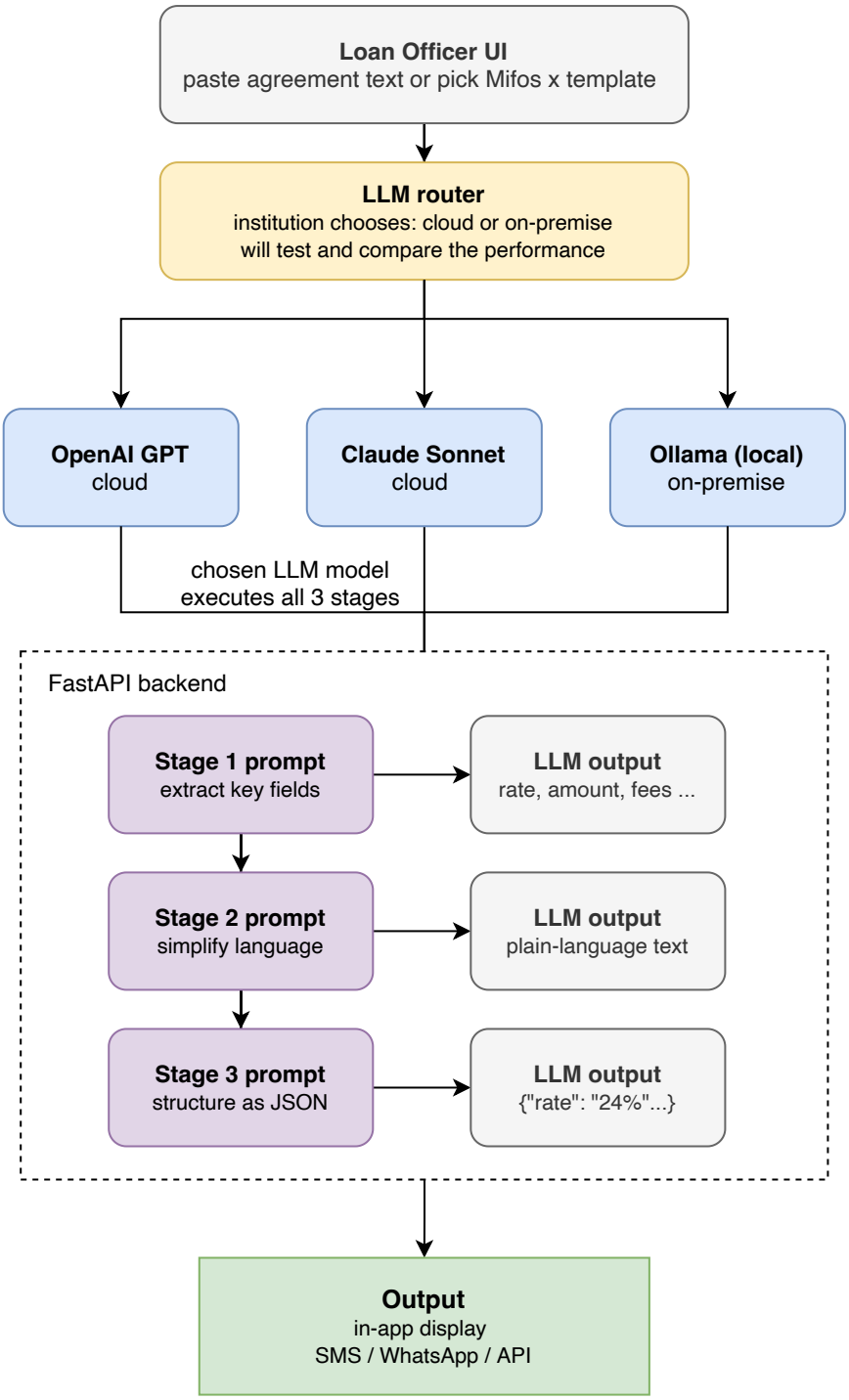
- Accepts a loan agreement text or Mifos X product template as input
- Uses a carefully engineered prompting pipeline to extract and explain six key fields in plain language
- Returns summaries formatted for in-app display, WhatsApp, and SMS
- Supports both cloud LLMs (OpenAI, Anthropic) and on-premise models (Ollama) for institutions with data sovereignty requirements
- Exposes a clean REST API for future integration into the Mifos X web app and mobile apps

### 2.3 Tech Stack

| Layer | Technology | Rationale |
|---|---|---|
| Backend service | Python 3.11, FastAPI | Lightweight, async, industry standard for ML services |
| LLM integration | OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet | Best-in-class instruction following; both available from day one |

| Layer | Technology | Rationale |
|---|---|---|
| On-premise LLM | Ollama (Llama 3, Mistral) | Data privacy for regulated MFIs; free, runs locally |
| Frontend UI | Streamlit | Rapid prototyping; ideal for loan officer-facing tools |
| Output formatting | Python (Jinja2 templates) | Structured SMS/WhatsApp output |
| API documentation | FastAPI built-in OpenAPI/Swagger | Auto-generated, always up to date |
| Testing | pytest, pytest-asyncio | Unit and integration testing for all endpoints |
| Version control | Git / GitHub | Standard; aligns with Mifos contribution workflow |
| Containerization | Docker | Consistent deployment across environments |

### 2.4 System Architecture & Workflow




The system has four layers:

- **Input layer** - Streamlit UI where a loan officer pastes or uploads a loan agreement, or selects a Mifos X product template.
- **LLM router** - selects the appropriate model based on the institution's deployment preference (could vs. on-premise). The chosen model executes all three prompt stages.
- **Three-stage prompt pipeline** (all all executed by the chosen LLM):
    - Stage 1 — Extract: pull six key fields from the agreement (loan amount, interest rate and type, repayment schedule, total cost of credit, late fees, default consequences)
    - Stage 2 — Simplify: rewrite extracted content at a low-literacy reading level, using concrete examples
    - Stage 3 — Structure: return a validated JSON object for consistent downstream handling

- **Output layer** — formatted summaries for in-app display, SMS/WhatsApp sharing, and a POST /summarize REST endpoint for Mifos X integration.

### 2.5 Prompt Engineering Example

**Input (raw agreement text):**

> "The borrower shall remit monthly installments calculated on a reducing balance methodology at a nominal annual rate of 24%, inclusive of administrative charges, over a tenure of 12 months."

**After Stage 1 (extraction):**

> Interest rate: 24% p.a., declining balance. Tenure: 12 months. Schedule: monthly.

**After Stage 2 (simplification):**

> "Your interest rate is 24% per year, calculated on your remaining balance — so as you pay down the loan, your interest charges decrease each month."

**After Stage 3 (JSON output):**

```
{
  "loan_amount": "TBD",
  "interest_rate": "24% per year, declining balance",
  "repayment_schedule": "12 monthly payments",
  "total_cost_of_credit": "calculated at runtime",
  "late_fee": "TBD from agreement",
  "default_consequence": "TBD from agreement"
}
```

### 2.6 Research Phase (Weeks 9-10)

Prior work shows RAG and multi-agent architectures can improve LLM summarization on domain-specific documents. If evaluation data shows prompt engineering hitting an accuracy ceiling — for example, if models hallucinate interest calculations — will explore:

- **RAG pipeline**: retrieve Mifos X product configuration rules as grounding context
- **Multi-agent approach**: one agent extracts, a second agent verifies against the original document

This phase is evidence-driven, not speculative. I enter it only if evaluation shows a measurable need.

**Research references:** Research Papers: https://github.com/Niuniu-Li-229/2026-GSoC-Mifos/tree/main/Contract%20Summarization%20with%20LLMs/Research%20Papers

### 2.7 Project Milestones & Timeline

| Period | Milestone | Deliverables |
|---|---|---|
| Community bonding (May 1–26) | Setup & planning | Mifos X local environment, annotated sample loan dataset, finalized prompt strategy with mentors |

| Period | Milestone | Deliverables |
|---|---|---|
| Weeks 1–2 (May 27–June 9) | Foundation | Streamlit UI, text parser, OpenAI + Anthropic API integration, repo structure |
| Weeks 3–4 (June 10–23) | Core service | Extraction + simplification prompts, JSON output parser, initial accuracy testing |
| Weeks 5–6 (June 24–July 7) | On-premise support | Ollama integration, begin systematic LLM comparison on annotated dataset |
| Weeks 7–8 (July 8–21) | Output & multilingual | SMS/WhatsApp formatter, multilingual exploration, **midterm evaluation** |
| Weeks 9–10 (July 22–Aug 4) | Research phase | Evaluate prompt ceiling; explore RAG / multi-agent if needed; document findings |
| Weeks 11–12 (Aug 5–18) | Evaluation & API | Full LLM evaluation report, REST API finalization, OpenAPI documentation |
| Week 13 (Aug 19–26) | Wrap-up | Code cleanup, final documentation, demo, GSoC final report submission |

## 2.8 Expected Outcomes & Impact

**Technical Outcomes**

- A working, open-source Python service deployable by any Mifos X instance
- A documented prompt library covering loan agreement summarization use cases
- An evidence-based evaluation report comparing GPT-4o, Claude, and Ollama with scoring across accuracy, completeness, readability, and cost
- A `POST /summarize` REST endpoint ready for integration into Mifos X web or mobile apps

**Social Impact**

- Loan officers can generate plain-language client summaries in seconds, reducing reliance on manual explanation
- Clients at low-literacy levels gain a genuine opportunity to understand what they are agreeing to — directly supporting responsible finance
- MFIs in low-connectivity or regulated environments can deploy the on-premise Ollama option with no data leaving their infrastructure
- The evaluation report provides a reusable decision framework for future AI projects in the Mifos ecosystem

## 2.9 Challenges & Mitigation

| Challenge | Mitigation |
|---|---|
| LLM hallucination on financial figures | Stage 1 extraction prompt uses strict JSON schema with validation; Stage 3 cross-checks extracted numbers against input text |

| Challenge | Mitigation |
|---|---|
| Flat-rate vs. declining-balance conflation | Explicit prompt instruction with worked examples; evaluation dataset includes both types with known correct outputs |
| Low-literacy readability is hard to measure | Use Flesch-Kincaid score as a quantitative proxy; additionally test on sample summaries with non-technical reviewers |
| Multilingual quality is uneven across models | Scope multilingual as exploratory; document model-specific performance honestly rather than overclaiming |
| On-premise Ollama models underperform cloud models | Evaluation will surface this explicitly; recommendation report will give honest tradeoff guidance |
| Mifos X template formats vary across deployments | Build parser to handle both raw text input and structured template fields; test on multiple template formats |

## 2.10 Testing, Documentation & Deployment

**Testing Strategy**

- **Unit tests (pytest):** each prompt stage tested independently with fixture inputs; JSON parser tested against malformed LLM responses
- **Integration tests:** end-to-end flow (raw text → summary) for each supported LLM using FastAPI TestClient
- **Evaluation framework:** annotated dataset of 20–30 sample loan agreements with ground-truth summaries; automated scoring on accuracy, completeness, and Flesch-Kincaid readability
- **Target:** >90% field-level accuracy for cloud models; >80% for Ollama

**Documentation**

- README with quickstart and environment setup
- Prompt library documentation with design rationale for each stage
- OpenAPI/Swagger reference auto-generated by FastAPI
- Evaluation report as a standalone markdown document

**Deployment**

- Dockerfile for containerized deployment
- Environment variable configuration for API keys and model selection
- Supports local development (Ollama), cloud API, and Docker-based production

## 2.11 Future Scope & Scalability

- **Direct Mifos X integration:** The `POST /summarize` endpoint is designed to be called directly from the loan origination workflow in the Mifos X web app or Android client, requiring minimal frontend changes
- **Fine-tuned model:** A dataset of high-quality loan summaries generated during this project could be used to fine-tune a smaller, faster model specifically for MFI use cases

- **Expanded document types:** The same pipeline architecture applies to other Mifos documents — savings product terms, insurance disclosures, group lending agreements
- **Voice output:** Text-to-speech integration would extend accessibility to clients who cannot read at all — a natural next step given Mifos's voice agent work in 2025
- **Feedback loop:** Loan officers rating summary quality in the UI would create a continuous improvement dataset over time

---

## 3. Any resources you will need to undertake the project:

- OpenAI API access — already have an active account
- Anthropic (Claude) API access — already have an active account
- Ollama for local model inference — free, runs locally, no cost

---

## 4. Why are you the right person for this project?

Most applicants will need to learn what loan agreements contain and what matters to borrowers. I already know this from years of professional practice.

At IFC (World Bank Group), I spent two years reading, modeling, and summarizing complex financial risk documents for non-specialist audiences — capital adequacy reports, stress test results, risk disclosures. Writing clear summaries of dense financial documents for senior management is directly the skill this project requires at a prompt engineering level.

I have used Python professionally for data pipelines and reporting automation at Citigroup, and I have immediate access to both OpenAI and Anthropic APIs. I can begin substantive work on the evaluation framework from day one.

Working at IFC also gave me direct exposure to the financial inclusion challenge Mifos is solving. I have seen how inaccessible financial language creates real barriers for underserved borrowers, which makes this project personally meaningful beyond the technical challenge.

---

## 5. Current area of study:

Master of Science in Computer Science (CS Align program), Khoury College of Computer Science, Northeastern University — Spring 2026, first year. Previously: MS in Finance (STEM-certified), Simon Business School, University of Rochester (2020).

---

## 6. Contact information and preferred method of contact:


- Email: wanjingyang29@gmail.com
- Slack: Wanjing Yang — preferred for day-to-day communication
- LinkedIn: linkedin.com/in/wyang37
- Timezone: PST (UTC-7)


---

## 7. Career goals:

I am transitioning from financial risk management into software engineering and ML/AI, with a focus on applied AI in fintech and data science. My goal is to leverage my finance domain expertise alongside emerging CS skills to work on real-world problems at the intersection of AI and financial services — ideally in roles that have measurable social or economic impact. GSoC with Mifos is a pivotal step: it would give me my first open source contribution, hands-on LLM engineering experience, and a concrete project that bridges both worlds.

---

## 8. Links to source code or websites:

GitHub: https://github.com/Niuniu-Li-229/2026-GSoC-Mifos/tree/main/Contract%20Summarization%20with%20LLMs

---

## 9. Slack nickname:

Wanjing Yang — introduced myself in #wg-ai-for-all and received a personal response from Priyanshu Tiwari.

---

## 10. Contributions to other open source projects outside of Mifos:

No prior open source contributions. This is my first cycle. I am actively working to make my first contribution to the relevant Mifos repository before the deadline.

---

## 11. Have you interacted with any of our mentors?

Yes — Priyanshu Tiwari responded to my introduction in the #wg-ai-for-all Slack channel and invited me to share my proposal for review via DM.

---

## 12. Previous experience with Angular/Java/Spring/Hibernate/MySQL/Android/DevOps:

- **Java:** Basic level — currently studying in NEU CS Align program
- **MySQL/SQL:** Proficient — used extensively at IFC and Citigroup for data pipelines and reporting
- **Angular/Spring/Hibernate/Android/DevOps:** Limited exposure — not directly required for this project, which is Python-based

---

## 13. Other commitments this summer:

Full-time GSoC commitment. I am enrolled in the NEU CS Align program and may take one course during the summer term, which would not conflict with GSoC hours. No internship or employment commitments planned.

---

## 14. Which Mifos open source projects have you deployed?

- **Mifos X:** In progress — currently setting up local development environment following the Getting Started Guide. Will update the screenshot before final submission.
- Payment Hub EE: N/A
- Mifos Gazelle: N/A
- Mobile Applications: N/A

---

## 15. Detail how you installed them and provide a screenshot:

Currently setting up Mifos X locally using Docker following the Apache Fineract Getting Started Guide. Will include screenshot of running environment before final submission.

---

## 16. Patches or source code submitted to Mifos Projects:

No submissions yet. I am actively working on a first contribution to the community-ai repository, which is directly relevant to this proposal. I will update this before the March 31 deadline.

---

## 17. What motivates you most about working with the Mifos Initiative?

At IFC, I worked on financial sustainability projects for institutions serving clients in developing economies. I saw firsthand that the barrier between a borrower and a fair loan often comes down to whether they can understand the document in front of them. Mifos is building the infrastructure to serve the world's 2 billion unbanked — and this project addresses one of the most human parts of that challenge: whether a client actually understands what they are agreeing to. That specificity of impact, applied through open-source software, is what makes Mifos different from any other GSoC organization I considered.

---

## 18. Previously participated in Google Summer of Code?

No — this is my first GSoC application.

---

## 19. Applying to multiple organizations this year?

No — Mifos is my only application.

---

## 20. First choice organization:

N/A — applying only to Mifos.

---

## 21. Previously participated in any other internship programme with Mifos?

No.